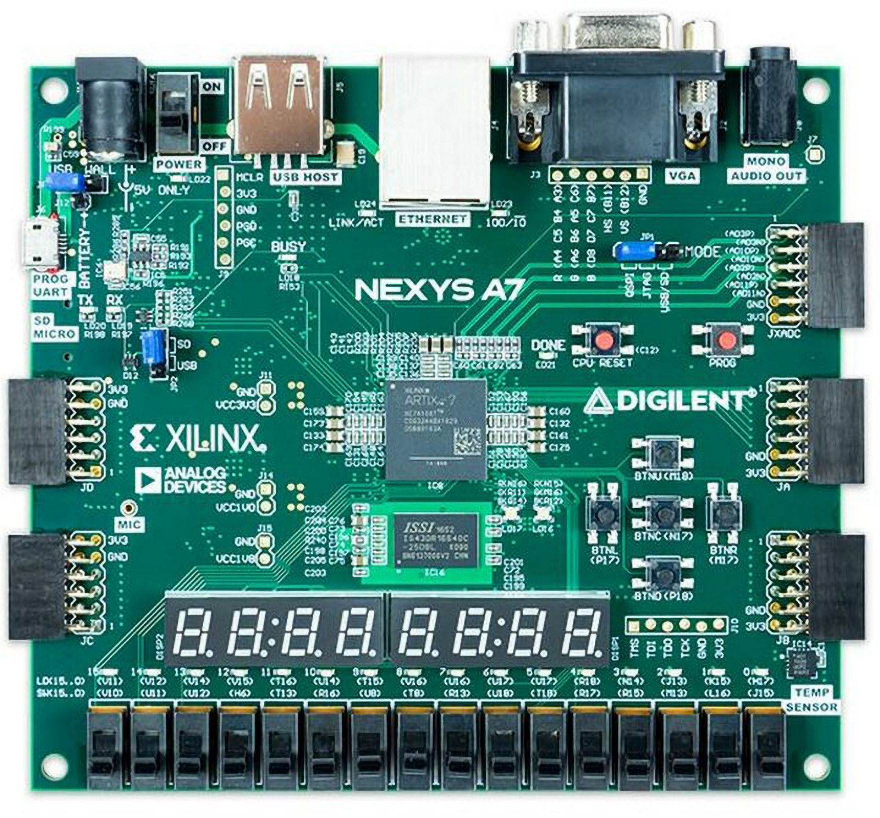
Streaming 2D Convolution Accelerator

By: Changwe Musonda, Kevin Wang, Emily Morales

Project overview

This project's objective is to **compare an FPGA A*** implementation against a **reference Python A*** under identical maps, start/goal pairs, and heuristic settings. The FPGA core runs A* on a **16×16 occupancy grid** with **8-directional** moves and a **Manhattan** heuristic, reporting **path length** and **cycle count** for latency/speedup analysis versus Python. A lightweight **Python 2D visualizer** simply renders the FPGA testbench outputs to **confirm correctness** and make results easy to present.

Hardware: Nexys A7



Project Roadmap and Deliverables

Start: 10/27 |End: 11/14

Verilog Code

We will implement the line buffers, the sliding window generator, and the fixed-point MAC pipeline, including frame/line valid control, then smoke-test the datapath in simulation.

Start: 10/27 | End: 11/14

Python Code

We will build the image I/O, a golden floating-/integer-precision convolution, and a visualizer so both hardware and software flows can run and produce comparable outputs.

Start: 11/14 | End: 11/21

Testing

We will sweep multiple kernels and images, verify border handling, quantify fixed-point error against the golden model, and check that the stream never stalls.

Start: 11/21 | End: 11/23

Comparing

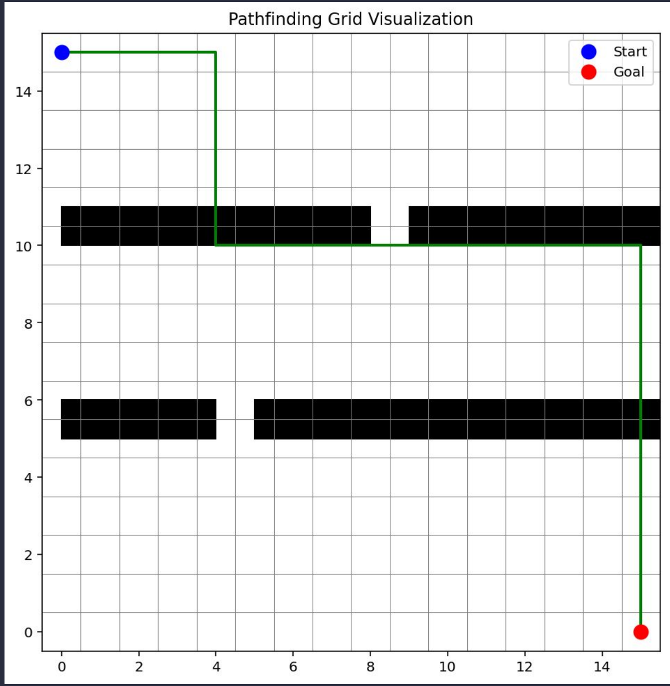
We will compare hardware and software results, highlighting image quality (e.g., PSNR), latency, resource use, and any differences in edge cases.

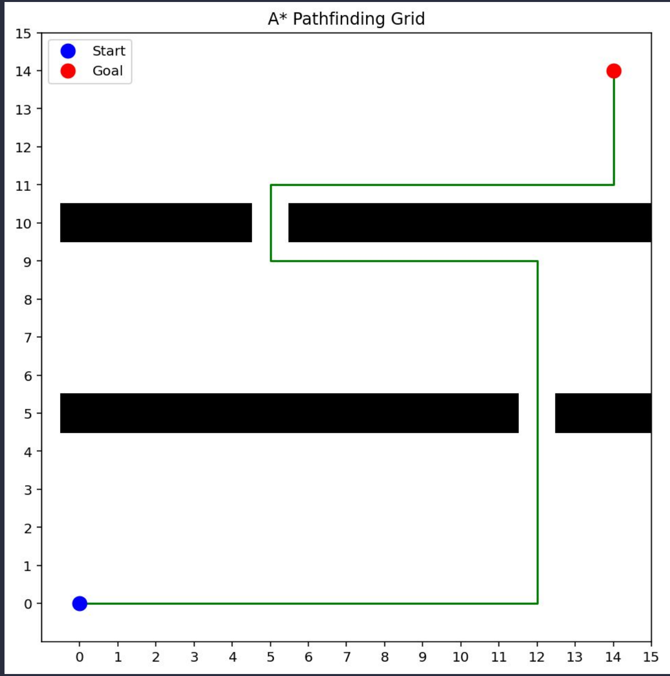
Start: 11/23 | End: 12/1

Results/Findings

We will decide which execution path is preferable for our constraints, record the findings, and finalize plots, tables, and the demo materials.

Example/Progress





Final Report Plan

KEY LEARNING 1

When the FPGA and Python A* use the same 4-connected moves, Manhattan heuristic, costs, and tie-break rules, they produce the same path and path length.

KEY LEARNING 2

Keeping data on-chip and doing simple operations each cycle lets the FPGA cut latency (cycles) versus Python; the open-set design sets how well this scales to bigger grids.

KEY LEARNING 3

The Python 2D visualizer confirms the FPGA path matches Python on each map, while the reported cycle count and path length make the speedup and correctness easy to judge.

Any questions? Ask away!

Thank you